KARADENIZ TECHNICAL UNIVERSITY — GRADUATE SCHOOL OF NATURAL AND APPLIED SCIENCES

Week 3 Laboratory Sheet

Search I: Systematic Search — Backtracking, Ordering Heuristics and CP-SAT Search Parameters

Assoc. Prof. Tolga Berber · https://avesis.ktu.edu.tr/tberber

Estimated time: 2 hours

Step 12 steps · 6 checkpoints

Requirements: Terminal/shell access (macOS/Linux: Terminal, Windows: PowerShell), the course's shared Python 3.12 virtual environment from Week 1 (already carrying `ortools`), a text editor

1 2 3 ✓ CP1 4 5 ✓ CP2 6 ✓ CP3 7 8 ✓ CP4 9 10 11 ✓ CP5 12 ✓ CP6

Learning phases: 1 · Pre-instruction 2 · Content 3 · Participation 4 · Assessment 5 · Follow-through

Why It Matters 1 · Pre-instruction

Every solver command you have typed so far this course has quietly run something you have never watched happen: a systematic loop that chooses a variable, chooses a value, and backtracks the instant a choice goes bad. Today you build and trace that loop by hand on a small over-constrained scheduling instance, watch two ordering heuristics change how much of a much larger search tree CP-SAT actually explores, catch a genuinely silent configuration bug that makes a forced search strategy do nothing at all, and close by tuning that same handful of parameters on a problem you pick yourself.

Prior Knowledge 1 · Pre-instruction

This sheet exercises all four units of this week's chapter: backtracking search and the search tree (Unit 1), variable/value ordering heuristics — MRV, the degree heuristic, dom/wdeg and LCV (Unit 2), forward checking and MAC (Unit 3), and CP-SAT's decision-strategy, worker-count and time-limit parameters (Unit 4). It also leans on two earlier weeks: Week 1's N-queens paper model and first CP-SAT models, and Week 2's local consistency and the AC-3 propagation fixpoint. Keep the chapter open beside this sheet — each section below points to the unit it exercises.

Use this sheet as follows: in each step, complete the task described under `What will you do?`, then compare your own result against the `Expected result` box to confirm you are on track before moving on. Steps follow the chapter's four units in order, and a `Checkpoint` every few steps lets you confirm everything so far is working before you continue.

1 Setup: Confirming the Shared Environment

Step 1: **Confirm this week needs no new package**

What will you do? With the course's shared virtual environment `kod_ornekleri/.venv` active, you will print the installed `ortools` version to confirm the same package Week 1 installed is still there — this week is CP-SAT only, so no new package install is needed.

Why? This week reuses Week 1's `ortools` install unchanged (no Gurobi, no MiniZinc this week); confirming the version before you start is what makes every `ciktikutusu` box below reproducible on your own machine.

Terminal

```
$ cd kod_ornekleri
$ source .venv/bin/activate
$ python -c "import ortools; print(ortools.__version__)"
```

Expected result: The command prints `9.14.6206`, the exact version Week 1 pinned for the whole course.

Expected Output

```
9.14.6206
```

Tip

If this command raises `ModuleNotFoundError`, your virtual environment is not active in this terminal — re-run `source .venv/bin/activate` from inside `kod_ornekleri` before continuing.

macOS / Linux

`source .venv/bin/activate` adds `(.venv)` to your shell prompt; every `python` command in the rest of this sheet then resolves to this environment's interpreter automatically.

Windows

In PowerShell, activate with `.venv\Scripts\Activate.ps1` instead.

2 Unit 1 — Backtracking and the Search Tree

(See Course Notes Week 3, Unit 1 — Backtracking and the Search Tree.)

Step 2: **Formulate a fresh over-constrained scheduling CSP**

What will you do? A small lab shares one rare calibration technician among three instruments, P, Q and R, each of which needs to be booked into one of two blocks this week (Morning, Afternoon). Because the same technician must be present for every booking, no two of P, Q and R may share a block. On paper, write this as a formal CSP triple (variables, domains, constraints) exactly as Week 1's formalism does, fix the branching order P, Q, R and the value order Morning-then-Afternoon (matching Unit 1's own 4-queens trace), and state — before tracing anything — whether the domain size and the number of mutually constrained variables already hint at trouble.

Why? Writing out a fresh CSP triple from a plain description, without a paper model handed to you first, is what turns following Unit 1's 4-queens formalization into building one yourself — and fixing the branching and value order now is what makes the trace in the next step fully determined, exactly as Unit 1's own worked example fixes `Q1,Q2,Q3,Q4` and ascending values before tracing a single branch.

Expected result:

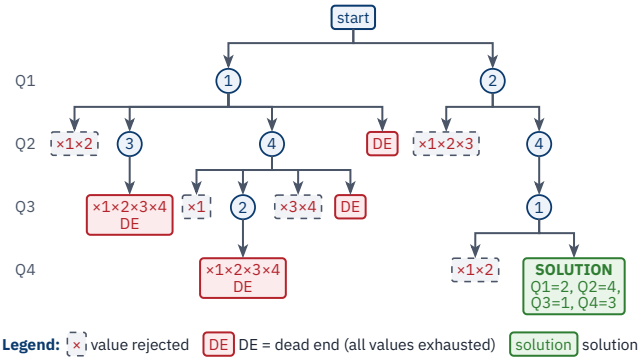
Variables: P, Q, R; domain for each: $\{\text{Morning}, \text{Afternoon}\}$; constraints: $P \neq Q$, $Q \neq R$, $P \neq R$ (every pair, since the technician cannot be in two blocks at once). Three variables are pairwise constrained to take different values, but the shared domain has only two elements — by the pigeonhole principle, no assignment can satisfy all three constraints at once. The full backtracking trace in the next step is what *proves* this formally, the way Unit 1's own completeness argument requires: a CSP is only correctly declared infeasible once the root itself becomes a dead end, not merely because a shortcut argument makes it look impossible.

Step 3: Complete the backtracking trace and count the dead ends

What will you do? Continuing the previous step's P, Q, R instance, the $P = \text{Morning}$ branch is traced for you below; complete the $P = \text{Afternoon}$ branch yourself, on paper, in exactly the same style, then report the total number of dead ends across the whole tree and state whether the search correctly declares the CSP infeasible.

Given (the $P = \text{Morning}$ branch): $P = \text{Morning}$: $Q = \text{Morning}$ rejected (same value as P); $Q = \text{Afternoon}$ accepted. $R = \text{Morning}$ rejected ($P \neq R$, same as P); $R = \text{Afternoon}$ rejected ($Q \neq R$, same as Q) — every value of R fails, **dead end 1**, backtrack to Q. Q's domain is now exhausted (Morning rejected outright, Afternoon led to a dead end), so $P = \text{Morning}$ itself becomes **dead end 2**; backtrack to the root.

Why? Unit 1's own hand trace works this exact procedure on 4-queens; repeating it here, on an instance small enough to finish completely by hand, is what turns following a worked dead-end count into running the whole tree to its conclusion yourself — including the one case 4-queens never shows you: a root that becomes a dead end and a CSP correctly declared infeasible.



Expected result:

$P = \text{Afternoon}$: $Q = \text{Morning}$ accepted ($P \neq Q$); $R = \text{Morning}$ rejected ($Q \neq R$, same as Q); $R = \text{Afternoon}$ rejected ($P \neq R$, same as P) — every value of R fails, **dead end 3**, backtrack to Q. $Q = \text{Afternoon}$ rejected ($P \neq Q$, same as P). Q's domain is now exhausted, so $P = \text{Afternoon}$ itself becomes **dead end 4**. P's own domain is now exhausted (both values led to dead ends) — the root itself becomes a dead end, so the search correctly reports the CSP **infeasible**: exactly **four** dead ends total, matching the pigeonhole argument from the previous step.

Tip

The figure above is Unit 1's own 4-queens tree, not this P, Q, R instance — use it only as a template for how a search tree is drawn (root, branches, dead-end leaves); your own P, Q, R tree has just two first-level branches, both dead, and no solution leaf at all.

Checkpoint — verify before moving on

You have formulated a fresh CSP entirely from a plain description and traced its backtracking search to completion by hand, finding exactly four dead ends and a correctly declared infeasibility. If both match the `Expected result` boxes above, you have exercised Unit 1's completeness argument on an instance 4-queens never demonstrates — move on to Unit 2.

3 Unit 2 — Variable and Value Ordering Heuristics

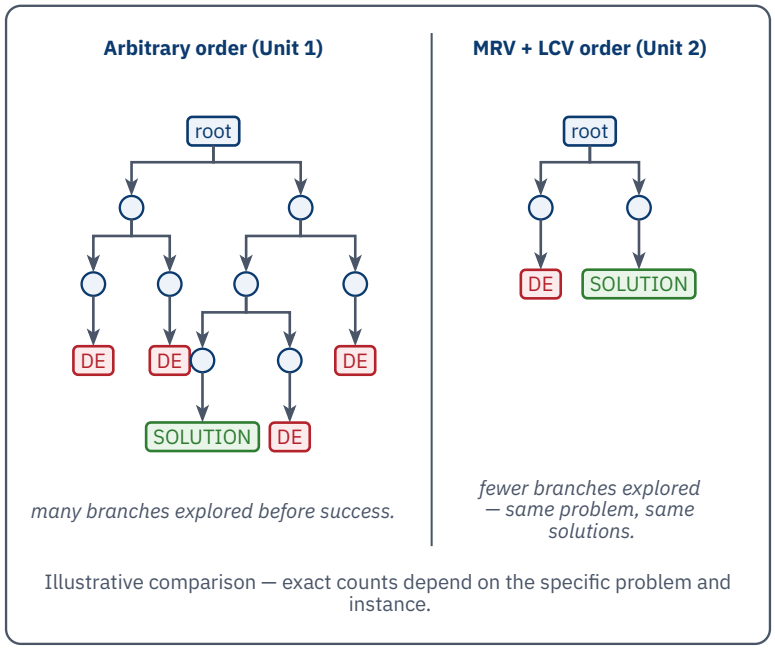
(See Course Notes Week 3, Unit 2 — Variable and Value Ordering Heuristics.)

Step 4: Apply MRV by hand to a Petersen-graph coloring snapshot

What will you do? This week's coded problem, the Petersen graph (10 nodes, 15 edges, chromatic number 3, every node touching exactly 3 edges), is coloured with one variable per node, domain $\{0, 1, 2\}$. Suppose node 0 is coloured first (a fixed, arbitrary order, exactly Unit 1's baseline) with colour 0. Its three neighbours — nodes 1, 4 and 5 — each lose colour 0 from their domain, narrowing them to $\{1, 2\}$ (size 2); every other node keeps its full domain $\{0, 1, 2\}$ (size 3). On paper, apply MRV to decide which node the search should branch on next, and state whether the degree heuristic is needed to break a tie here.

Why? Unit 2's own practice task applies MRV to a three-variable snapshot from an abstract graph-coloring CSP; repeating the same reasoning on this week's own Petersen instance, whose structure you will run as real code in the very next step, is what turns following Unit 2's tie-break example into predicting what CP-SAT's `CHOOSE_MIN_DOMAIN_SIZE` strategy will do before you ever run it.

Expected result: MRV selects among nodes 1, 4 and 5 (domain size 2), ignoring every node still at domain size 3, regardless of degree. Because the Petersen graph is vertex-transitive, nodes 1, 4 and 5 all have exactly the same degree (3) as every other node, so the degree heuristic cannot break the tie between them on this snapshot alone — either an arbitrary tie-break or a history-sensitive heuristic like dom/wdeg would be needed to choose among the three.



Tip

The figure above is a schematic illustration of the general arbitrary-order-versus-MRV+LCV effect on tree shape, not a drawing of this specific Petersen snapshot — use it to recall *why* a smaller domain gets picked first, before the next step shows you the effect on a real, much larger search tree.

Step 5: Run graph_coloring.py: default order versus MRV+LCV, for real

What will you do? You will run `graph_coloring.py` twice — once under CP-SAT's default search, once under `AddDecisionStrategy` forcing `CHOOSE_MIN_DOMAIN_SIZE` (CP-SAT's MRV analogue) and `SELECT_MIN_VALUE` (standing in for LCV's intuition) — and compare the printed `NumBranches` and `NumConflicts` between the two runs.

Why? This is Unit 2's abstract ordering argument, running as real code for the first time this week: the same domain-narrowing effect you just reasoned about by hand on node 0's neighbours is exactly what `CHOOSE_MIN_DOMAIN_SIZE` exploits at every single node of the real 10-variable search, not just the first one.

The variable-selection and value-selection strategy, forced onto every one of the ten colour variables:

```
54 model.AddDecisionStrategy(
55     colours, cp_model.CHOOSE_MIN_DOMAIN_SIZE, cp_model.SELECT_MIN_VALUE
56 )
57 solver = cp_model.CpSolver()
58 solver.parameters.search_branching = cp_model.FIXED_SEARCH
59 solver.parameters.num_search_workers = 1
60 solver.parameters.random_seed = 42
61 status = solver.Solve(model)
62 print_search_stats(solver, status)
```

Terminal

```
$ cd hafta03
$ python modeller/graph_coloring.py
$ python -c "
import sys; sys.path.insert(0, 'modeller')
import graph_coloring
graph_coloring.petersen_solve_ordered()
"
```

Expected result: The default run reports `NumBranches: 69`, `NumConflicts: 1`; the MRV+LCV-forced run reports `NumBranches: 65`, `NumConflicts: 0` — a modest but real reduction in explored search effort on the exact same problem, matching Unit 2's "same solutions, often a smaller explored tree" claim. Your own `WallTime` values may differ slightly from the ones below; the branch and conflict counts should match exactly, since both runs fix `random_seed=42` and `num_search_workers=1`.

Expected Output

```
Status: OPTIMAL
NumBranches: 69
NumConflicts: 1
WallTime: 0.0009
```

Expected Output

```
Status: OPTIMAL
NumBranches: 65
NumConflicts: 0
WallTime: 0.0008
```

Tip

If your terminal is still at the course root, `cd kod_ornekleri` first, then `cd hafta03` — every remaining command in this sheet is run from inside `kod_ornekleri/hafta03`.

Checkpoint — verify before moving on

You have applied MRV by hand to a real Petersen-graph snapshot and confirmed, on the full 10-node problem, that forcing `CHOOSE_MIN_DOMAIN_SIZE` and `SELECT_MIN_VALUE` reduces `NumBranches` from 69 to 65 and `NumConflicts` from 1 to 0. If both match the `Expected result` boxes above, move on to Unit 3.

4 Unit 3 — Forward Checking and MAC

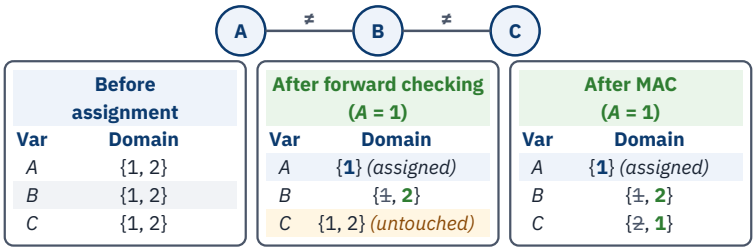
(See Course Notes Week 3, Unit 3 — Forward Checking and MAC.)

Step 6: Extend the chapter's forward-checking example to MAC, by hand

What will you do? Recall Unit 3's worked example: three variables A, B, C on a path ($A \neq B$, $B \neq C$, no direct A–C constraint), domains $A : \{1, 2\}$, $B : \{1, 2\}$, $C : \{1, 2\}$, with $A = 1$ assigned. Forward checking narrows B to $\{2\}$ but leaves C untouched at $\{1, 2\}$, since C is not A's direct

neighbour. On paper, continue the propagation the way MAC would: since B is now forced to $\{2\}$, re-check C's domain against the now-narrower B and state C's domain after MAC finishes.
Why? This is the one step Unit 3's own worked example stops short of — forward checking's one-hop update leaves C alone, but MAC keeps propagating past B; working out that second step by hand yourself, before checking it against the figure, is what turns stating that MAC is stronger than forward checking into tracing the extra propagation step yourself.

Forward checking versus MAC: domain updates after *A* = 1



Before assignment	
Var	Domain
A	{1, 2}
B	{1, 2}
C	{1, 2}

After forward checking (*A* = 1)	
Var	Domain
A	{**1**} *(assigned)*
B	{~~1~~, **2**}
C	{1, 2} *(untouched)*

After MAC (*A* = 1)	
Var	Domain
A	{**1**} *(assigned)*
B	{~~1~~, **2**}
C	{~~2~~, **1**}

Forward checking only updates *A*'s direct neighbour *B*, leaving *C*'s domain untouched (amber flag); MAC continues propagating the singleton domain of *B* and also narrows *C*, which forward checking never reaches.

Expected result: With B forced to $\{2\}$, the constraint $B \neq C$ removes value 2 from C's domain, leaving $C : \{1\}$ — a narrowing forward checking never finds at this step, since C is two hops away from the just-assigned A, not one. Check your own before/after domain tables against the figure above: $A : \{1\}$ throughout, $B : \{1, 2\} \to \{2\}$ under both techniques, $C : \{1, 2\} \to \{1, 2\}$ under forward checking but $C : \{1, 2\} \to \{1\}$ under MAC.

Tip

Every one of MAC's extra propagation steps here is exactly Week 2's AC-3 fixpoint — MAC does not need a new algorithm, only a new moment to call the algorithm you already built: after every single assignment during search, not just once before search begins.

Checkpoint — verify before moving on
You have extended Unit 3's forward-checking example to MAC entirely by hand and confirmed the extra narrowing on C ($\{1, 2\} \to \{1\}$) against the chapter's own domain-table figure. You are ready for the week's CP-SAT parameters, which sit on top of exactly this kind of propagation.

5 Unit 4 — Search Parameters in CP-SAT

(See Course Notes Week 3, Unit 4 — Search Parameters in CP-SAT.)

Step 7: **Run the incomplete graph_coloring_lab.py and find the silent bug**

What will you do? A starter file, `kod_ornekleri/hafta03/lab/graph_coloring_lab.py`, reuses the Petersen model's variables and constraints unchanged and calls `AddDecisionStrategy` with the same `CHOOSE_MIN_DOMAIN_SIZE` plus `SELECT_MIN_VALUE` pair you just ran in Unit 2 — but its solver setup is missing one line, marked with a `TODO`. Run the file exactly as given and compare its printed statistics against both of Unit 2's own runs, not just the one you expect it to match.
Why? Unit 4's own subtopics state plainly that `AddDecisionStrategy` only takes effect once `solver.parameters.search_branching` is switched to `FIXED_SEARCH`; running a model that forces a strategy without that switch, on a problem where you already know both the default and the correctly-forced numbers, is what makes "the forced strategy is silently ignored" concrete rather than a warning you take on faith.
The incomplete solver setup:

```
model.AddDecisionStrategy(
    colours, cp_model.CHOOSE_MIN_DOMAIN_SIZE, cp_model.SELECT_MIN_VALUE
)
solver = cp_model.CpSolver()
# TODO: AddDecisionStrategy above only takes effect once CP-SAT's
# search mode is switched away from automatic search. Set
# solver.parameters.search_branching to the right enum value here.
solver.parameters.num_search_workers = 1
```

Terminal

```
$ python lab/graph_coloring_lab.py
```

Expected result: A status of `OPTIMAL` with `NumBranches: 69` and `NumConflicts: 1` — not the `65`/`0` you saw from the correctly-forced run in Unit 2, but exactly the *default* run's numbers instead. The `AddDecisionStrategy` call above ran without error and changed nothing at all.

Expected Output

```
Status: OPTIMAL
NumBranches: 69
NumConflicts: 1
WallTime: 0.0007
```

Common Pitfall

A result that runs without error and looks like a legitimate solve is tempting to accept at face value. It should not be: with no `search_branching = cp_model.FIXED_SEARCH` line, CP-SAT silently falls back to its own automatic search and the `AddDecisionStrategy` call above is never consulted at all — matching Unit 2's default numbers exactly is the tell, not an error message.

Step 8: **Add the missing line and verify the fix**

What will you do? In `lab/graph_coloring_lab.py`, where the `TODO` comment says to, add `solver.parameters.search_branching = cp_model.FIXED_SEARCH` immediately before the existing `num_search_workers` line. Then re-run the script.

Why? This is Unit 4's own configuration recipe, applied here as a hands-on repair rather than only read in prose — fixing a one-line omission and watching a silently-ignored strategy start actually steering the search is what turns stating the `FIXED_SEARCH` requirement into knowing exactly what happens when it is missing, because you just watched it happen.

Terminal

```
$ python lab/graph_coloring_lab.py
```

Expected result: `NumBranches: 65` and `NumConflicts: 0` — now matching Unit 2's correctly-forced run exactly, confirming the fix.

Expected Output

```
Status: OPTIMAL
NumBranches: 65
NumConflicts: 0
WallTime: 0.0006
```

Tip

The fixed line reads `solver.parameters.search_branching = cp_model.FIXED_SEARCH` — every `*_solve_ordered` function inside `modeller/graph_coloring.py` already sets this line; the starter file above deliberately left it out.

Checkpoint — verify before moving on

You caught a forced decision strategy that silently did nothing, fixed the missing `FIXED_SEARCH` line yourself, and watched `NumBranches` drop from 69 to 65 as a direct result. You now have direct, hands-on evidence for exactly why Unit 4 insists this line is not optional.

Step 9: Preview num_search_workers on the same Petersen model

What will you do? You will run `petersen_solve_workers` twice on the same Petersen model, once with `num_search_workers=1` and once with `num_search_workers=8`, and compare only `WallTime` between the two — CP-SAT's built-in parallel portfolio, previewed here as a knob and its visible effect, not explained further.

Why? Per this week's scope, *why* multiple workers help — and when they do not — is Week 4's subject in full; this step stops at "here is the knob and what it visibly does" on a problem small enough that the difference is easy to read off directly.

Terminal

```
$ python -c "
import sys; sys.path.insert(0, 'modeller')
import graph_coloring
print('num_search_workers=1'); graph_coloring.petersen_solve_workers(1)
print(); print('num_search_workers=8'); graph_coloring.petersen_solve_workers(8)
"
```

Expected result: Both runs report the same `NumBranches: 69` and `NumConflicts: 1` — the search itself does the same work either way on a problem this small — but the 8-worker run's `WallTime` is not faster; on an instance this tiny, the overhead of coordinating 8 threads outweighs any benefit, a genuine result worth noticing before Week 4 explains when parallel search actually pays off.

Expected Output

```
num_search_workers=1
Status: OPTIMAL
NumBranches: 69
NumConflicts: 1
WallTime: 0.0008
```

```
num_search_workers=8
Status: OPTIMAL
NumBranches: 69
NumConflicts: 1
WallTime: 0.0024
```

Caution

Do not conclude from this one tiny instance that more workers never help — Week 4 revisits this exact parameter on problems large enough for parallel search to pay off. Here, the lesson is narrower: the knob exists, it is easy to set, and its effect is not always the one you might guess.

Step 10: **Run magic_square.py: default versus MRV+LCV, for the first time in code**

What will you do? You will run `magic_square.py` twice — default search, then the same `CHOOSE_MIN_DOMAIN_SIZE` plus `SELECT_MIN_VALUE` strategy you have now used twice — on the 4×4 magic square CSP Week 1 formulated only on paper, and compare `NumBranches` and `NumConflicts` between the two runs.

Why? This is Week 1 Unit 3's magic square getting its first CP-SAT code, and the same ordering comparison you ran on graph coloring, applied to a structurally different problem (16 variables, one global `AllDifferent` plus 10 sum constraints instead of 15 pairwise `!=` constraints) — confirming the ordering effect is not an artifact of one specific problem shape.

The 16 cell variables and every constraint, including the magic constant's 10 equal-sum constraints:

```
29 model.AddAllDifferent([cell for row in cells for cell in row])
30 for r in range(N):
31     model.Add(sum(cells[r]) == MAGIC_CONSTANT)
32 for c in range(N):
33     model.Add(sum(cells[r][c] for r in range(N)) == MAGIC_CONSTANT)
34 model.Add(sum(cells[i][i] for i in range(N)) == MAGIC_CONSTANT)
35 model.Add(sum(cells[i][N - 1 - i] for i in range(N)) == MAGIC_CONSTANT)
```

Terminal

```
$ python modeller/magic_square.py
$ python -c "
import sys; sys.path.insert(0, 'modeller')
import magic_square
magic_square.magicsquare_solve_ordered()
"
```

Expected result: The default run reports `NumBranches: 3107`, `NumConflicts: 53`, and prints a completed 4×4 grid whose every row, column and both diagonals sum to 34; the MRV+LCV-forced run reports `NumBranches: 3067`, `NumConflicts: 33` — again a real, if modest, reduction on a considerably larger search space than Petersen's.

Expected Output

```
Status: OPTIMAL
NumBranches: 3107
NumConflicts: 53
WallTime: 0.0119
[8, 1, 10, 15]
[3, 13, 6, 12]
[14, 4, 11, 5]
[9, 16, 7, 2]
```

Expected Output

```
Status: OPTIMAL
NumBranches: 3067
NumConflicts: 33
WallTime: 0.0114
```

Tip

`magicsquare_solve_ordered` does not print the grid, only the search statistics — both runs solve the identical CSP, so they are guaranteed to find *a* valid magic square (not necessarily the same one), and only the search effort needed to find it is being compared here.

Step 11: **Run langford.py: default, ordered and time-limited, for real**

What will you do? You will run `langford.py` three times — default search, `CHOOSE_MIN_DOMAIN_SIZE` plus `SELECT_MIN_VALUE` forced, and default search capped at a `max_time_in_seconds` far below what this instance needs — on the Langford($n = 7$) CSP, and report which of the first two runs' `NumBranches` is smaller.

Why? Langford pairs was chosen specifically because its search-tree size is dramatically sensitive to variable and value order; running all three configurations yourself, on real captured numbers rather than a prediction, is the only way to answer Unit 4's own comparison question honestly — this week's style rules forbid asserting which configuration wins before the numbers exist.

The two position variables per value and the "positions differ by $k + 1$" constraint:

```
31 model.AddAllDifferent([entry for pair in positions for entry in pair])
32 for k in range(1, N + 1):
33     first, second = positions[k - 1]
34     model.Add(second - first == k + 1)
```

Terminal

```
$ python modeller/langford.py
$ python -c "
import sys; sys.path.insert(0, 'modeller')
import langford
langford.langford_solve_ordered()
langford.langford_time_limited()
"
```

Expected result: The default run reports `NumBranches: 564`, `NumConflicts: 21`; the ordered run reports `NumBranches: 843`, `NumConflicts: 0`. On this particular instance and this particular forced strategy, the *default* run's `NumBranches` is the smaller of the two — the opposite direction from both Petersen and the magic square above, a real counter-example to "forcing MRV+LCV always shrinks the tree." The time-limited run, capped far below the roughly 2 ms this instance needs, reports `Status: UNKNOWN` and `NumBranches: 0`: a genuinely honest result of a budget spent entirely on model transfer and presolve, before the search takes its first branch.

Expected Output

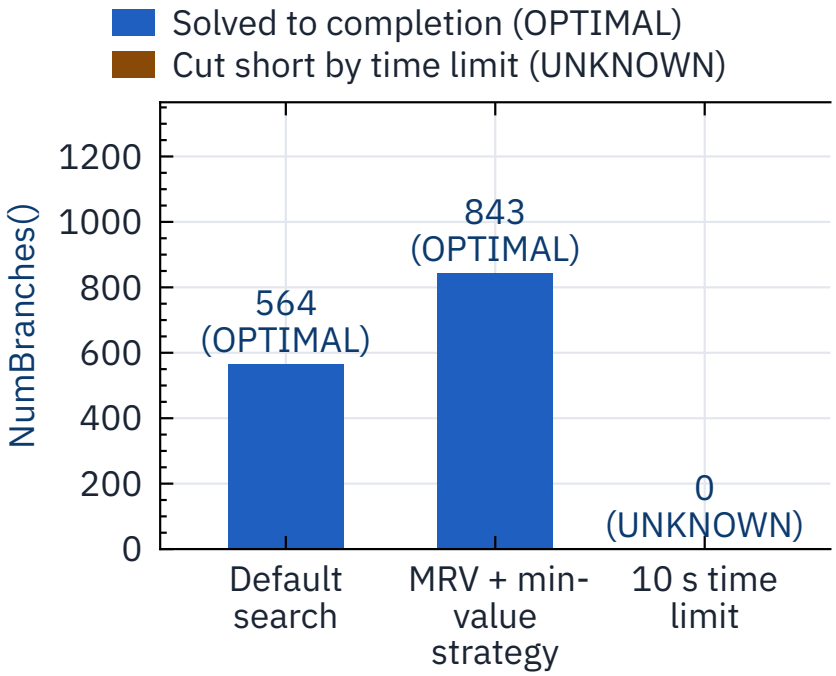
```
Status: OPTIMAL
NumBranches: 564
NumConflicts: 21
WallTime: 0.0020
```

Expected Output

```
Status: OPTIMAL
NumBranches: 843
NumConflicts: 0
WallTime: 0.0025
```

Expected Output

```
Status: UNKNOWN
NumBranches: 0
NumConflicts: 0
WallTime: 0.0010
```



Real CP-SAT statistics from langford7-default-stats.txt, langford7-ordered-stats.txt, langford7-time-limited-stats.txt — mixed result, no configuration highlighted as a winner.

Common Pitfall

`Status: UNKNOWN` with `NumBranches: 0` looks like a crash or a misconfigured model, but it is neither — it is `max_time_in_seconds` doing exactly what it promises: return whatever the solver has found so far once the clock runs out, even if that is nothing at all yet. Always check `StatusName` before trusting any other statistic from a time-limited run.

Checkpoint — verify before moving on

You caught and fixed a silent `FIXED_SEARCH` omission, previewed `num_search_workers`' visible-but-not-always-helpful effect, confirmed the MRV+LCV ordering effect on a second, larger problem (magic square), and read three real Langford($n = 7$) configurations — including one honest `UNKNOWN` result. Every parameter Unit 4 introduced has now run on real code, on real numbers.

6 Capstone — Configure Your Own Search on a Problem You Pick

Step 12: Compare at least two decision strategies plus a time limit, on a problem of your choice

What will you do? Choose one of this week's three problems — graph coloring, magic square or Langford pairs. Write, in a new file (no starter file this time), a short script that imports that problem's `build_variables` and `add_constraints` functions (and `search_stats.print_search_stats`) from `modeller/`, then solves the same model under at least two distinct `AddDecisionStrategy` configurations you have not already run this week (for example, a different pairing of variable- and value-selection enums than `CHOOSE_MIN_DOMAIN_SIZE`

plus `SELECT_MIN_VALUE`), plus one run under a deliberately tight `max_time_in_seconds`. Record every run's real `NumBranches`, `NumConflicts` and `Status`, and write two or three sentences comparing them.

Why? This is this week's integrative step: H1–H4 gave you the vocabulary and the individual parameters one at a time; choosing your own configurations, on a problem you pick, and reading the real difference in search effort off the solver's own statistics — rather than a configuration and an instance this sheet already chose for you — is what H5 asks of you, the same "apply it yourself, on your own choice of instance" pattern every earlier week's capstone in this course has used.

Terminal

```
$ python -c "
import sys; sys.path.insert(0, 'modeller')
from ortools.sat.python import cp_model
from graph_coloring import build_variables, add_constraints
from search_stats import print_search_stats

model = cp_model.CpModel()
colours = build_variables(model)
add_constraints(model, colours)
model.AddDecisionStrategy(colours, cp_model.CHOOSE_FIRST, cp_model.SELECT_MAX_VALUE)
solver = cp_model.CpSolver()
solver.parameters.search_branching = cp_model.FIXED_SEARCH
solver.parameters.num_search_workers = 1
solver.parameters.random_seed = 42
status = solver.Solve(model)
print_search_stats(solver, status)
"
```

Expected result: One legitimate real run, shown here on the Petersen graph with `CHOOSE_FIRST` (Unit 1's baseline order) plus `SELECT_MAX_VALUE` (the opposite of every value-selection rule used earlier this week): `NumBranches: 66`, `NumConflicts: 0` — between the unordered default's 69 and MRV+LCV's 65, confirming that *some* of Unit 2's benefit here comes from the value order alone, not only from MRV. Your own script's exact numbers will depend on the problem and configurations you chose; what matters is that every number you report is a real printed `NumBranches`/`NumConflicts`/`Status`, not a guess.

Expected Output

```
Status: OPTIMAL
NumBranches: 66
NumConflicts: 0
WallTime: 0.0006
```

Tip

A second configuration worth trying on the same Petersen model, `CHOOSE_MIN_DOMAIN_SIZE` plus `SELECT_MAX_VALUE`, reports `NumBranches: 65`, `NumConflicts: 0` — the same branch count MRV+LCV reached with `SELECT_MIN_VALUE`, a useful data point for your own written comparison: on this instance, which variable is chosen next seems to matter more than which value is tried first.

Common Pitfall

Forgetting `solver.parameters.search_branching = cp_model.FIXED_SEARCH` here reproduces exactly the silent bug you diagnosed earlier in this section — a script that runs without error and quietly reports the default configuration's numbers under a different label. Check that your own forced runs' numbers actually differ from the unforced default before you write your comparison.

Checkpoint — verify before moving on

Your own script compared at least two decision-strategy configurations plus a time-limited run, on a problem you chose, using only real `NumBranches`/`NumConflicts`/`Status` values — matching the same variables-constraints-decision-strategy-solve pattern you have now run five times today, the last time entirely on parameters of your own choosing.

Troubleshooting

Symptom	Likely cause	Fix
`ModuleNotFoundError: No module named 'ortools'`	The virtual environment is not active in this terminal	Re-run `source .venv/bin/activate` (or `Activate.ps1` on Windows) from inside `kod_ornekleri`
Running any `modeller/*.py` or `lab/*.py` script prints `No such file or directory`	The command was run from the wrong working directory	Run `pwd` and confirm you are inside `kod_ornekleri/hafta03` before running any script
Your fixed `graph_coloring_lab.py` still reports `NumBranches: 69`	The new line was added after `solver.Solve(model)` was already called, or misspelled as `search_branch` instead of `search_branching`	Re-check that `solver.parameters`' `search_branching` field reads `cp_model.FIXED_SEARCH` and sits before `solver.Solve(model)`
Your own capstone script reports `NumBranches` identical to the unforced default, even after adding `AddDecisionStrategy`	`FIXED_SEARCH` was not set, the same silent bug diagnosed earlier in Unit 4	Re-read the `ortakhata` box in this section's capstone step and add the missing line
A time-limited run reports a `Status` other than `UNKNOWN` even though you set a very small `max_time_in_seconds`	The instance you chose is small enough that presolve and the first few branches finish before the clock runs out	Lower `max_time_in_seconds` further, or choose the magic square or Langford instance, whose search takes longer than Petersen's
`pytest -q` reports failures unrelated to your own edits	`modeller/` files were edited directly instead of the `lab/` starter file	Restore `modeller/*.py` from the course repository; only `lab/graph_coloring_lab.py` is meant to be edited this week

Transfer — Where Else? 5 · Follow-through

The three problems you ran today — graph coloring, magic square and Langford pairs — are not disposable lab work: Week 4 reuses `search_stats.py` unchanged as its own starting point for reading search statistics on local search and restarts, and Week 15 revisits exactly these decision-strategy, worker-count and time-limit parameters when a much harder model refuses to solve at all. More generally, the habit you practiced five times today — state the variables and constraints first, predict roughly what ordering or look-ahead should help before you run anything, then let the solver's own `NumBranches`, `NumConflicts` and `WallTime` confirm or correct that prediction — is the same habit every solver you will ever configure rewards, whether it is CP-SAT here, Gurobi's branch-and-bound from Week 7 onward, or a solver you meet for the first time in your own thesis work. A forced configuration that silently does nothing, as you saw twice today, is far easier to catch when you already know what the honest numbers should look like.